ASSIGNMENT-15.4

2303A52481

SINDHU KATAKAM

BATCH-41

Task 1: Initial API Setup for a Service Platform

PROMPT:

Write a simple Flask application in Python to set up a basic API for a service platform. Create a root endpoint /status that returns a JSON response indicating that the API is running (for example: {"status": "API is running"}). Ensure the server runs locally without errors and can be tested using a browser or an API testing tool like Postman. Include clear code and brief comments explaining how the API works and how to run it.

CODE:

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/status')
def status():
    return jsonify({"status": "API is live"})

if __name__ == '__main__':
    # In a Colab environment, '0.0.0.0' makes the server accessible within Colab's network.
    # 'debug=True' provides helpful debugging information.
    app.run(host='0.0.0.0', port=5000, debug=True)
```

```python
import requests
import time

# Give the server a moment to start
time.sleep(3)

try:
  response = requests.get('http://127.0.0.1:5000/status')
  print(response.json())
except requests.exceptions.ConnectionError:
  print("Could not connect to the Flask server. Make sure the Flask app cell is running.")
```

OUTPUT:

```
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

JUSTIFICATION:

This task helped me learn how a backend actually starts and runs in a real project. By creating a simple /status API, I understood how a server responds to requests and confirms

that the system is working. I also learned how to run the Flask server and check the output, which is an important first step before building bigger features. Completing this task gave me a clear basic idea of how APIs work in real applications.

Task 2: Create API to Add and View Resources

PROMPT:

Write a Python Flask application that manages service requests stored in memory (using a list or dictionary). Create a GET endpoint /requests to return all service requests and a POST endpoint /requests to add a new request with fields like title and priority. Ensure that initially the GET request returns an empty list, and after adding a new request using POST (for example: {"title": "Internet issue", "priority": "High"}), it returns a confirmation along with the added data. Also include simple comments in the code and instructions on how to run and test the API.

CODE:

```
#!pip install Flask # Uncomment and run this line first if Flask is not installed

import os
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for service requests
# Each request will be a dictionary with 'id', 'title', 'priority'
requests_db = []
next_request_id = 1

@app.route('/requests', methods=['GET'])
def get_requests():
    """Returns all service requests."""
    return jsonify(requests_db)

@app.route('/requests', methods=['POST'])
def add_request():
    """Adds a new service request."""
    global next_request_id
    data = request.get_json()
```

```python
    global next_request_id
    data = request.get_json()

    if not data or not all(key in data for key in ['title', 'priority']):
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': next_request_id,
        'title': data['title'],
        'priority': data['priority']
    }
    requests_db.append(new_request)
    next_request_id += 1

    return jsonify({
        'message': 'Service request added successfully',
        'request': new_request
    }), 201

print("Starting Flask application...")
app.run(host='0.0.0.0', port=5000)
```

OUTPUT:

```
Starting Flask application...
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
```

JUSTIFICATION:

The goal of this task was to create a simple web application using Python's Flask framework. This application allows us to manage service requests by providing two basic functionalities: first, we can retrieve a list of all existing service requests, and second, we can add new service requests, specifying their title and priority. This setup demonstrates how to build a basic API that interacts with in-memory data, providing a foundation for understanding more complex web services.

Task 3: Modify Existing Resource Using AI Assistance

PROMPT:

Write a Python Flask application that extends an existing in-memory service request API. Use AI to generate a PUT endpoint /requests/<int:index> that updates a service request based on its index. The request should allow updating fields like title and priority. Ensure that if a valid index is provided, the API updates the request and returns a response like {"message": "Request updated", "request": {...}}. If an invalid index is given, handle it properly by

returning a clear error message with HTTP status code 404. Include simple comments in the code and instructions on how to test the endpoint.

CODE:

```
import os
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for service requests
# Each request will be a dictionary with 'id', 'title', 'priority'
requests_db = []
next_request_id = 1

@app.route('/requests', methods=['GET'])
def get_requests():
    """Returns all service requests."""
    return jsonify(requests_db)

@app.route('/requests', methods=['POST'])
def add_request():
    """Adds a new service request."""
    global next_request_id
    data = request.get_json()

    if not data or not all(key in data for key in ['title', 'priority']):
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
```

```python
        'id': next_request_id,
        'title': data['title'],
        'priority': data['priority']
    }
    requests_db.append(new_request)
    next_request_id += 1

    return jsonify({
        'message': 'Service request added successfully',
        'request': new_request
    }), 201

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    """Updates an existing service request by ID."""
    data = request.get_json()

    if not data:
        return jsonify({'error': 'No data provided for update'}), 400

    # Find the request by ID
    target_request = None
    target_index = -1
    for i, req in enumerate(requests_db):
        if req['id'] == request_id:
            target_request = req
```

```python
            target_index = i
            break

    if target_request is None:
        return jsonify({'error': 'Service request not found'}), 404

    # Update fields if provided
    if 'title' in data:
        requests_db[target_index]['title'] = data['title']
    if 'priority' in data:
        requests_db[target_index]['priority'] = data['priority']

    return jsonify(
        {
            'message': 'Service request updated successfully',
            'request': requests_db[target_index]
        }
    ), 200


print("Starting Flask application...")
app.run(host='0.0.0.0', port=5000)
```

OUTPUT:

```
Starting Flask application...
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
```

JUSTIFICATION:

The purpose of this extension was to enhance our simple service request application by adding the ability to modify existing requests. Previously, we could only view and create new requests. With the new 'PUT' endpoint, we can now update details like the title or priority of a specific request, identified by its ID. This makes the application more dynamic and practical, as it allows for corrections or changes to service requests after they have been initially submitted.

Task 4: Remove a Resource Safely

PROMPT:

Write a Python Flask application that adds a DELETE endpoint to an existing in-memory service request API. The endpoint `/requests/<int:index>` should remove a service request based on its index. Ensure that when a valid index is provided, the API deletes the request and returns a response with a confirmation message along with the deleted request details. If an invalid index is given, handle it properly by returning a clear error message without crashing the server. Include simple comments in the code and instructions on how to test the endpoint.

CODE:

```python
import os
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for service requests
# Each request will be a dictionary with 'id', 'title', 'priority'
requests_db = []
next_request_id = 1

@app.route('/requests', methods=['GET'])
def get_requests():
    """Returns all service requests."""
    return jsonify(requests_db)

@app.route('/requests', methods=['POST'])
def add_request():
    """Adds a new service request."""
    global next_request_id
    data = request.get_json()

    if not data or not all(key in data for key in ['title', 'priority']):
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': next_request_id,
```

```
        'title': data['title'],
        'priority': data['priority']
    }
    requests_db.append(new_request)
    next_request_id += 1

    return jsonify({
        'message': 'Service request added successfully',
        'request': new_request
    }), 201

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    """Updates an existing service request by ID."""
    data = request.get_json()

    if not data:
        return jsonify({'error': 'No data provided for update'}), 400

    # Find the request by ID
    target_request = None
    target_index = -1
    for i, req in enumerate(requests_db):
        if req['id'] == request_id:
            target_request = req
            target_index = i

            break

    if target_request is None:
        return jsonify({'error': 'Service request not found'}), 404

    # Update fields if provided
    if 'title' in data:
        requests_db[target_index]['title'] = data['title']
    if 'priority' in data:
        requests_db[target_index]['priority'] = data['priority']

    return jsonify(
        {
            'message': 'Service request updated successfully',
            'request': requests_db[target_index]
        }
    ), 200

@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    """Deletes a service request by ID."""
    global requests_db
    target_request = None
    target_index = -1
    for i, req in enumerate(requests_db):
        if req['id'] == request_id:
```

```python
        if req['id'] == request_id:
            target_request = req
            target_index = i
            break

    if target_request is None:
        return jsonify({'error': 'Service request not found'}), 404

    # Remove the request from the list
    deleted_request = requests_db.pop(target_index)

    return jsonify(
        {
            'message': f'Service request with ID {request_id} deleted successfully
            'deleted_request': deleted_request
        }
    ), 200


rint("Starting Flask application...")
pp.run(host='0.0.0.0', port=5000)
```

OUTPUT:

```
Starting Flask application...
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
```

JUSTIFICATION:

The purpose of this latest addition was to complete our service request management system by enabling the removal of requests. While we could previously view, create, and update requests, the absence of a delete function made the system incomplete. The new 'DELETE' endpoint allows users to permanently remove a specific service request using its unique ID, ensuring that the in-memory database accurately reflects only active and relevant requests. This capability is crucial for maintaining data hygiene and providing full CRUD (Create, Read, Update, Delete) functionality.

Task 5: AI-Assisted API Documentation and Readability

PROMPT:

Write a Python Flask application for a service request API and improve its readability using AI assistance. Add clear and meaningful docstrings and inline comments to all endpoints (GET, POST, PUT, DELETE), explaining the purpose, HTTP methods, input parameters, and response

formats. Ensure that the code is easy for another developer to understand without reading the full implementation. Optionally, integrate automatic API documentation using tools like Swagger or Flask-RESTX to display endpoint paths, methods, and sample request/response formats. Include instructions on how to run and access the documentation UI.

CODE:

```
#!pip install Flask flask-restx # Install Flask and Flask-RE

import os
from flask import Flask, request, jsonify
from flask_restx import Api, Resource, fields # Import Flask-RESTx components

app = Flask(__name__)

# Initialize Flask-RESTx API
# The 'doc' parameter specifies the URL path for the Swagger UI documentation.
api = Api(app,
          version='1.0',
          title='Service Request API',
          description='A simple API for managing service requests in memory with
          doc='/docs' # Swagger UI will be available at /docs
         )

# Create a namespace for our API endpoints to organize them in Swagger UI
ns = api.namespace('requests', description='Service Request operations')

# In-memory storage for service requests
# Each request will be a dictionary with 'id', 'title', 'priority'
requests_db = []
next_request_id = 1

# --- Flask-RESTx Models for API documentation ---
```

```python
# --- Flask-RESTx Models for API documentation ---

# Model for the input when creating or updating a service request
request_input_model = ns.model('RequestInput', {
    'title': fields.String(required=True, description='The title of the service re
    'priority': fields.String(required=True, description='The priority of the serv
})

# Model for the output representation of a service request
request_output_model = ns.model('RequestOutput', {
    'id': fields.Integer(readOnly=True, description='The unique identifier of the
    'title': fields.String(required=True, description='The title of the service re
    'priority': fields.String(required=True, description='The priority of the serv
})

# --- API Endpoints ---

@ns.route('/') # Defines the route for '/requests'
class RequestList(Resource):
    @ns.doc('list_requests') # Documentation for this specific endpoint operation
    @ns.marshal_list_with(request_output_model) # Specifies the response format as
    def get(self):
        # HTTP Method: GET
        # Response: 200 OK, returns a JSON array of all service requests.
        return requests_db

    @ns.doc('create_request') # Documentation for this speci
    @ns.expect(request_input_model, validate=True) # Specifies the expected input
    @ns.marshal_with(request_output_model, code=201) # Specifies response format f
    @ns.response(400, 'Validation Error') # Documents a possible error response
    def post(self):
        global next_request_id
        # Flask-RESTx automatically parses the JSON body into api.payload
        data = api.payload
        new_request = {
            'id': next_request_id,
            'title': data['title'],
            'priority': data['priority']
        }
        requests_db.append(new_request)
        next_request_id += 1

        # With @ns.marshal_with, return the data directly; Flask-RESTx will format
        return new_request, 201 # 201 Created status code

@ns.route('/<int:request_id>') # Defines the route for '/requests/{request_id}'
@ns.response(404, 'Service request not found') # Documents a common error for this
class Request(Resource):
    """Manages individual service requests (PUT, DELETE)."""

    @ns.doc('update_request') # Documentation for this specific endpoint operation
    @ns.expect(request_input_model, validate=True) # Expects input for updating
```

```python
@ns.marshal_with(request_output_model) # Specifies the response format
def put(self, request_id):
    data = api.payload

    if not data: # Check if any data is provided for update
        return {'message': 'No data provided for update'}, 400

    # Find the request by ID
    target_request = None
    target_index = -1
    for i, req in enumerate(requests_db):
        if req['id'] == request_id:
            target_request = req
            target_index = i
            break

    if target_request is None:
        # Return 404 Not Found if request_id does not exist
        return {'message': f'Service request with ID {request_id} not found'},

    # Update fields if provided in the request body
    if 'title' in data:
        requests_db[target_index]['title'] = data['title']
    if 'priority' in data:
```

```python
    @ns.doc('delete_request') # Documentation for this specific endpoint operation
    @ns.response(200, 'Service request deleted successfully') # Documents successf
    def delete(self, request_id):

        global requests_db # Declare as global to modify the list directly
        target_request = None
        target_index = -1
        for i, req in enumerate(requests_db):
            if req['id'] == request_id:
                target_request = req
                target_index = i
                break
        if target_request is None:
            return {'message': f'Service request with ID {request_id} not found'},

        deleted_request = requests_db.pop(target_index)
        return {
            'message': f'Service request with ID {request_id} deleted successfully
            'deleted_request': deleted_request
        }, 200 # 200 OK status code


print("Starting Flask application...")
app.run(host='0.0.0.0', port=5000)
```

OUTPUT:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

JUSTIFICATION:

This task helped me understand that writing code alone is not enough, it should also be easy for others to read and use. By adding docstrings and comments, I learned how to clearly explain what each API endpoint does, what inputs it takes, and what output it returns. This makes it easier for other developers to understand and work with the API without confusion. It also showed me the importance of proper documentation in real projects, where multiple people work together.